

agent-meow 双平台实施范围报告

计划 001-010 · 灵创K16 + 橘宝R16 对比路线图

灵创K16: AMD Ryzen AI MAX+ 395 (Strix Halo) · iGPU 96GB + NPU

橘宝R16: AMD Ryzen AI 9 HX 470 + RTX 5060 (8GB CUDA) + NPU

日期: 2026-08-04 · **状态**: 灵创K16 已推送, 橘宝R16 评估中

双平台硬件对比

组件	灵创K16 (AI MAX+ 395)	橘宝R16 (HX 470+5060)
CPU	16C/32T Zen5	12C/24T Zen5 (Gorgon Point)
iGPU	Radeon 8060S (RDNA 3.5)	Radeon 890M (RDNA 3.5)
iGPU 显存	96GB 统一内存	32GB 统一内存
dGPU	无	RTX 5060 Laptop (8GB GDDR7)
dGPU 计算	—	CUDA + Vulkan (Blackwell)
NPU	XDNA 2	XDNA 2 (~50 TOPS)
ROCm	7.1 活跃 (HIP 后端)	7.1 (iGPU Vulkan 优先)
CUDA	无	RTX 5060 原生支持
内存	32GB DDR5	32GB DDR5-5600

关键差异：灵创K16 靠 96GB 统一显存容纳大模型；橘宝R16 靠 RTX 5060 CUDA 加速。

双平台架构分工

灵创K16 (Strix Halo)

CPU: TTS+VAD+QAA网关

iGPU: LLM(38GB)+STT(Vulkan)

NPU: 未来STT

橘宝R16 (HX470 + RTX 5060)

CPU: TTS+VAD+QAA网关

iGPU: 轻量计算 (Radeon 890M)

dGPU: LLM(8GB)+STT(CUDA) ← 关键

NPU: 未来STT

灵创K16: LLM 和 STT 都在 iGPU (96GB 统一显存充裕)

橘宝R16: LLM 和 STT 在 RTX 5060 dGPU (CUDA 原生支持, 8GB GDDR7)

优势对比: 橘宝R16 的 RTX 5060 支持 CUDA → faster-whisper **直接可用**, 无需 whisper.cpp Vulkan 替代

计划 001–005：基础设施修复（双平台通用）

#	计划	状态	灵创K16	橘宝R16
001	db_models 路径	TODO	通用	通用
002	VIDEOS_SURFACE.md	TODO	通用	通用
003	Phase 4 runner	TODO	通用	通用
004	过时 voicebox	TODO	通用	通用
005	Voicebox 可靠性	DRAFT	被 006+008 取代	同

优先级：低于 QAA 语音迁移。代码卫生修复不依赖硬件平台。

计划 006+006b: QAA 网关 + 混合部署 (双平台通用)

已解决: 原 S2S 冷启动 90 秒问题已通过 QAA 网关 + GPU STT 解决。faster-whisper 仅 CUDA 的限制已绕过。

- 灵创K16: whisper.cpp Vulkan 替代方案已实现
- 橘宝R16: RTX 5060 **有 CUDA** → faster-whisper 直接可用!

指标	灵创K16	橘宝R16
在线预热	~ 0s (DashScope)	~ 0s (同)
离线 STT 方案	whisper.cpp Vulkan	faster-whisper CUDA
离线 STT 预热	~3s (Vulkan)	~ 1s (CUDA 原生)
成本	90天免费, 后 ~¥0.20/分	同

QAA v1.3.0 + DashScope, 每会话 provider 切换。风险: LOW • 工作量: S

计划 007：QAA 语音钩子 → MeowCat（双平台通用）

保留猫爪 UI，替换传输层 — 与硬件无关。

旧组件	新组件	动作
realtimeVoice.ts (221行)	QAA useRealtimeVoice.js	替换
s2s_proxy.py (233行)	QAA 网关	替换
猫爪按钮+波形	保留	不变

协议差异：QAA 用 GatewayClientEvent JSON vs 当前自定义二进制帧。风险: MED · 工作量: L

计划 008：GPU STT 加速 — 双平台不同路径

灵创K16: whisper.cpp + Vulkan (`GGML_VULKAN=1`)

- 根因：faster-whisper 仅 CUDA，96GB iGPU 闲置
- 方案：whisper.cpp Vulkan 后端，STT 放到 Radeon 8060S
- 预热：CPU 60s → **GPU ~3s**

橘宝R16: faster-whisper + CUDA (RTX 5060 原生)

- **无需替代**！RTX 5060 支持 CUDA，faster-whisper 直接运行
- 预热：CPU 60s → **GPU ~1s** (CUDA 比 Vulkan 更快)
- 8GB GDDR7 充裕容纳 whisper-large-v3 (~1.5GB)

指标	灵创K16 (Vulkan)	橘宝R16 (CUDA)
STT 引擎	whisper.cpp	faster-whisper
GPU 后端	Vulkan	CUDA
STT 预热	~3s	~1s

计划 009+010: ACP → Hermes → Ollama 本地

灵创K16: Ollama + ROCm 7.1 (HIP 后端)

- 38GB qwen3.6:35b 在 96GB iGPU 显存, 占 40%
- `HIP_VISIBLE_DEVICES=0` 激活

橘宝R16: Ollama + CUDA (RTX 5060) 或 ROCm (Radeon 890M)

- 选用 **Qwen3.6-35B-A3B** (MoE, 3B 激活) — 与灵创K16 同模型
- MoE 仅 3B 激活 → 活跃层驻留 8GB dGPU, 256 专家分布 GPU+RAM
- **IQ3_XXS** (~13GB) 或 **Q4_K_M** (22GB) 量化, GPU offload 活跃层
- 替代: Radeon 890M + 32GB 统一内存可全量加载 (较慢)

指标	灵创K16	橘宝R16
LLM 模型	35B-A3B Q8_0 (38GB)	35B-A3B IQ3_XXS (~13GB)
推理位置	iGPU 96GB (ROCm)	dGPU 8GB+RAM (CUDA)
推理速度	~20-30 tok/s	~15-25 tok/s (MoE 3B 激活)

双平台优化栈对比

组件	灵创K16 引擎	灵创K16 位置	橘宝R16 引擎	橘宝R16 位置
LLM	Ollama+ROCm	iGPU 96GB	Ollama+CUDA	dGPU 8GB+RAM
STT	whisper.cpp+Vulkan	iGPU	faster-whisper+CUDA	dGPU
TTS	Kokoro-82M	CPU	Kokoro-82M	CPU
VAD	Silero	CPU	Silero	CPU
网关	QAA	CPU	QAA	CPU
代理OS	Hermes/Ollama	CPU+iGPU	Hermes/Ollama	CPU+dGPU
前端	React+Vite	浏览器	React+Vite	浏览器
NPU	未来 (winml)	NPU	未来 (winml)	NPU

灵创K16 优势： 96GB 显存全量加载 35B-A3B Q8_0，量化损失最小

橘宝R16 优势： CUDA 原生支持，STT 更快；MoE 3B 激活适合 8GB dGPU

依赖关系与执行顺序

阶段	灵创K16	橘宝R16	说明
1	006 + 008	006 + 008	并行 (008 路径不同)
2	006b + 007	006b + 007	依赖 006
3	009	009	依赖 006
4	010	010	依赖 009

关键差异：橘宝R16 的计划 008 更简单（`pip install faster-whisper` 即可，无需编译 whisper.cpp）。计划 010 使用同一 35B-A3B MoE 模型，但 IQ3 量化 + GPU/RAM 混合 offload。

已实现效果对比

指标	灵创K16 (优化前→已实现)	橘宝R16 (优化前→已实现)
语音预热	90s → ~0s	90s → ~0s
STT 预热	60s → ~3s (Vulkan)	60s → ~1s (CUDA)
LLM 模型	35B MoE Q8 (38GB)	35B MoE IQ3 (~13GB)
LLM 推理	iGPU ROCm 96GB	dGPU CUDA 8GB+RAM
GPU 利用率	0% → STT+LLM	0% → STT+LLM
云端依赖	必须 → 可选	必须 → 可选
成本	API → 零 (离线)	API → 零 (离线)
实现难度	MED (Vulkan 编译)	LOW (CUDA 原生)

双平台愿景：灵创K16 以 96GB 显存跑 35B-A3B MoE Q8_0（全量，质量优先）；橘宝R16 以 RTX 5060 CUDA 跑同一 35B-A3B MoE IQ3（混合 offload，速度+易用性优先）。两个平台使用同一模型架构，均实现零云端离线语音代理。